

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКЕ
(2025/2026 учебный год)

_____ Градсков Андрей Денисович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 24.06.2026 по 07.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Зинкин С.А.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

_____ Градсков Андрей Денисович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 24.06.2026 по 07.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Зинкин С.А.

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	2	24.06.2026 - 25.06.2026	
2	Подбор и изучение материала по теме работы	15	26.06.2026 – 28.06.2026	
3	Разработка алгоритма	43	01.07.2026 – 03.07.2026	
4	Описание алгоритма и программы	18	03.07.2026 – 04.07.2026	
5	Тестирование	5	04.07.2026 – 05.07.2026	
6	Получение и анализ результатов	10	05.07.2026 – 08.07.2026	
7	Оформление отчёта	15	05.07.2026 – 07.07.2026	
	Общий объём часов	108		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Градсков Андрей Денисович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 24.06.2026 по 07.07.2026

Кафедра «Вычислительная техника»

Градсков А.Д. выполнял практическое задание «Сортировка Шелла». На первоначальном этапе были изучен и проанализирован алгоритм сортировки Шелла, был выбран метод решения и язык программирования С, на котором была написана программа сортировки массива данным способом. Также, написал алгоритм генерации чисел в массив. Протестировал и отладил программу. Оформил отчёт.

Бакалавр Градсков А.Д. "___" _____ 2026г.

Руководитель Зинкин С.А. "___" _____ 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Градсков Андрей Денисович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 24.06.2026 по 07.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Градсков А.Д. решил следующие задачи: создание алгоритма сортировки Шелла, анализ работы алгоритма, сравнение существующих методов сортировки.

За период выполнения практики были освоены основные понятия и технологии сортировки Шелла. Во время выполнения работы Градсков А.Д. показал себя ответственным, добросовестным учеником, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике, программированию и сортировке.

За выполнение работы Градсков А.Д. заслуживает оценки «_____».

Руководитель практики д.т.н., профессор, Зинкин С.А. « » 2026 г.

Содержание

Введение	2
1 Постановка задачи.....	3
1.1 Достоинства алгоритма сортировки вставками.....	4
1.2 Недостатки алгоритма сортировки вставками.....	4
1.3 Типичные сценарии применения данного алгоритма	5
2 Выбор решения.....	6
3 Описание программы	6
4. Схемы программы	9
4.1 Блок-схема программы	9
4.2 Блок-схема алгоритма	10
5 Тестирование программы	12
6 Отладка.....	24
7 Совместная разработка	26
Заключение.....	30
Список используемой литературы.....	31
Приложение А. Листинг программы	32

Введение

Сортировка данных на сегодняшний день, при современном развитии компьютерных технологий, является одним из наиболее распространенных процессов современной обработки данных. Задачи на сортировку данных встречаются очень часто в различных профессиональных сферах деятельности.

Алгоритмы сортировки образуют отдельный класс алгоритмов, применяются практически во всех задачах обработки информации. При этом они настолько тесно связаны друг с другом, что образуют отдельный класс алгоритмов. Алгоритмы сортировки, как правило, применяются с целью осуществления последующего более быстрого поиска. Например, трудно пользоваться базами данных или словарями, если бы записи в них не были бы упорядочены.

Важность сортировки основана на том факте, что на ее примере можно показать многие основные фундаментальные приемы и методы построения алгоритмов. Сортировка является хорошим примером огромного разнообразия алгоритмов, которые выполняют одну и ту же задачу. Кроме того, многие из них имеют определенные преимущества друг перед другом. За счет усложнения алгоритма можно добиться существенного увеличения эффективности и быстродействия программы по сравнению с более простыми методами. Как правило, термин «сортировка» понимают как процесс перестановки объектов некоторого множества в определенном порядке.

Сортировка Шелла является усовершенствованным вариантом сортировки вставками. В отличие от простых алгоритмов, данный метод позволяет значительно сократить время выполнения за счет сравнения элементов, находящихся на определенном расстоянии друг от друга (шаге). По мере работы алгоритма шаг уменьшается, пока не будет выполнена финальная сортировка вставками. Сортировка Шелла демонстрирует высокую эффективность на массивах средней и большой размерности и часто

используется в качестве базового алгоритма в более сложных гибридных методах сортировки.

В рамках данной учебной практики была поставлена задача самостоятельной реализации алгоритма сортировки Шелла на языке программирования Си. Особенностью разрабатываемой программы является наличие графического интерфейса пользователя (на базе Win32 API), поддержка чтения и записи данных в формате CSV, а также проведение сравнительного анализа производительности реализованного алгоритма со стандартной библиотечной функцией `qsort`.

1 Постановка задачи

Поставленная задача: необходимо заполнить массив из n -ого количества элементов случайными числами, записать данные элементы в отдельный файл. После этого выполнить сортировку над данными, находящимися в массиве, записать отсортированные данные в другой файл, посчитать время выполнения и количество перестановок значений массива при сортировке.

Использовать сервис GitHub для совместной работы. Создать и выложить коммиты, характеризующие действия, выполненные каждым участником бригады.

Оформить отчет по проведенной практике.

1.1 Достоинства алгоритма сортировки Шелла

- не требует выделения дополнительной оперативной памяти для работы (сортировка на месте), в отличие от сортировки слиянием;
- демонстрирует высокую скорость работы на массивах среднего размера по сравнению с простыми квадратичными алгоритмами (пузырьком, выбором, вставками);
- простая реализация алгоритма, так как он является логичным усовершенствованием сортировки вставками;
- алгоритм удобен и эффективен для работы с частично отсортированными наборами данных.

1.2 Недостатки алгоритма сортировки Шелла

- не требует выделения дополнительной оперативной памяти для работы (сортировка на месте), в отличие от сортировки слиянием;
- демонстрирует высокую скорость работы на массивах среднего размера по сравнению с простыми квадратичными алгоритмами (пузырьком, выбором, вставками);
- простая реализация алгоритма, так как он является логичным усовершенствованием сортировки вставками;

- алгоритм удобен и эффективен для работы с частично отсортированными наборами данных.

1.3 Типичные сценарии применения данного алгоритма

- не требует выделения дополнительной оперативной памяти для работы (сортировка на месте), в отличие от сортировки слиянием;

- демонстрирует высокую скорость работы на массивах среднего размера по сравнению с простыми квадратичными алгоритмами (пузырьком, выбором, вставками);

- простая реализация алгоритма, так как он является логичным усовершенствованием сортировки вставками;

- алгоритм удобен и эффективен для работы с частично отсортированными наборами данных.

2 Выбор решения

Нашей бригадой было выбрано вести разработку в среде Microsoft Visual Studio на языке C.

Для написания данной программы будет использован язык программирования Си. Этот язык является распространённым языком программирования. При разработке языка Си был принят компромисс между низким уровнем языка ассемблера и высоким уровнем других языков. Си – это язык программирования общего назначения, хорошо известный своей эффективностью, экономичностью и переносимостью. Указанные преимущества Си обеспечивают хорошее качество разработки почти любого вида программного продукта.

Microsoft Visual Studio — это программная среда по разработке приложений для ОС Windows, как консольных, так и с графическим интерфейсом.

Для удобства совместной разработки был использован сервис GitHub. GitHub — сервис для контроля версий проекта. Проект динамично разрабатывается, регулярно расширяя функционал и возможности. Ведется активная работа с пожеланиями пользователей .

3 Описание программы

При запуске программы выводится графическое меню из шести пунктов:

- а) Загрузить CSV;
- б) Сортировать (Шелл+ qsort);
- в) Сохранить CSV;
- г) Размер;
- д) Сгенерировать;
- е) Поле выбора

Для оценки производительности и сравнения алгоритмов в программе реализован блок, отвечающий за сортировку и сбор метрик. В начале определяется структура для хранения статистики, которая включает в себя количество сравнений, перестановок (сдвигов) и процессорное время выполнения.

Перед началом тестирования исходный массив копируется в отдельную область памяти. Это необходимо для того, чтобы применить к копии стандартный библиотечный алгоритм и корректно сравнить время работы двух методов на абсолютно одинаковых входных данных.

```
static void copy_array(const long* src, long*
dst, int len)
{
    memcpy(dst, src, (size_t)len * sizeof(long));
}
```

Основным алгоритмом, реализуемым в программе, является сортировка Шелла. Она представляет собой модификацию сортировки вставками: сначала элементы сравниваются и упорядочиваются с большим шагом, затем шаг уменьшается, пока не станет равен единице, что гарантирует полную упорядоченность массива.

В данной программе используется последовательность шагов Кнута. Изначально вычисляется максимальный шаг, после чего в цикле он уменьшается в три раза. На каждом этапе элемент сравнивается с предыдущими в его подмассиве и сдвигается на нужную позицию. В процессе работы программа осуществляет детальный подсчет количества сравнений и

СДВИГОВ ЭЛЕМЕНТОВ.

```
h = 1;
while (h < len / 3) {
    h = 3 * h + 1;
}

while (h >= 1) {
    for (i = h; i < len; i++) {
        temp = array[i];
        j = i;
        while (j >= h) {
            stats->comparisons++;
            if (array[j - h] > temp) {
                array[j] = array[j - h];
                stats->swaps++;
                j -= h;
            }
            else {
                break;
            }
        }
        if (j != i) array[j] = temp;
    }
    h /= 3;
}
```

Для сравнения быстродействия копия массива сортируется с помощью стандартной библиотечной функции `qsort`. Программа оборачивает её вызов в отдельную функцию замера времени, что позволяет объективно оценить эффективность сортировки Шелла на конкретных типах данных.

```
// Функция сравнения двух элементов для qsort
static int compare_long(const void* a, const void* b) {
    long va = *(const long*)a;
    long vb = *(const long*)b;
    if (va < vb) return -1;
    if (va > vb) return 1;
    return 0;
}

// Замер времени для стандартной qsort
start = clock();
qsort(array, (size_t)len, sizeof(long), compare_long);
end = clock();
stats->time_seconds = (double)(end - start) / CLOCKS_PER_SEC;
```

По завершении работы алгоритмов в структуре статистики сохраняется процессорное время, затраченное на выполнение сортировки, а также накопленные счетчики сравнений и перестановок для дальнейшего вывода в графическом интерфейсе и сохранении в файл.

4. Схемы программы

4.1 Блок-схема программы

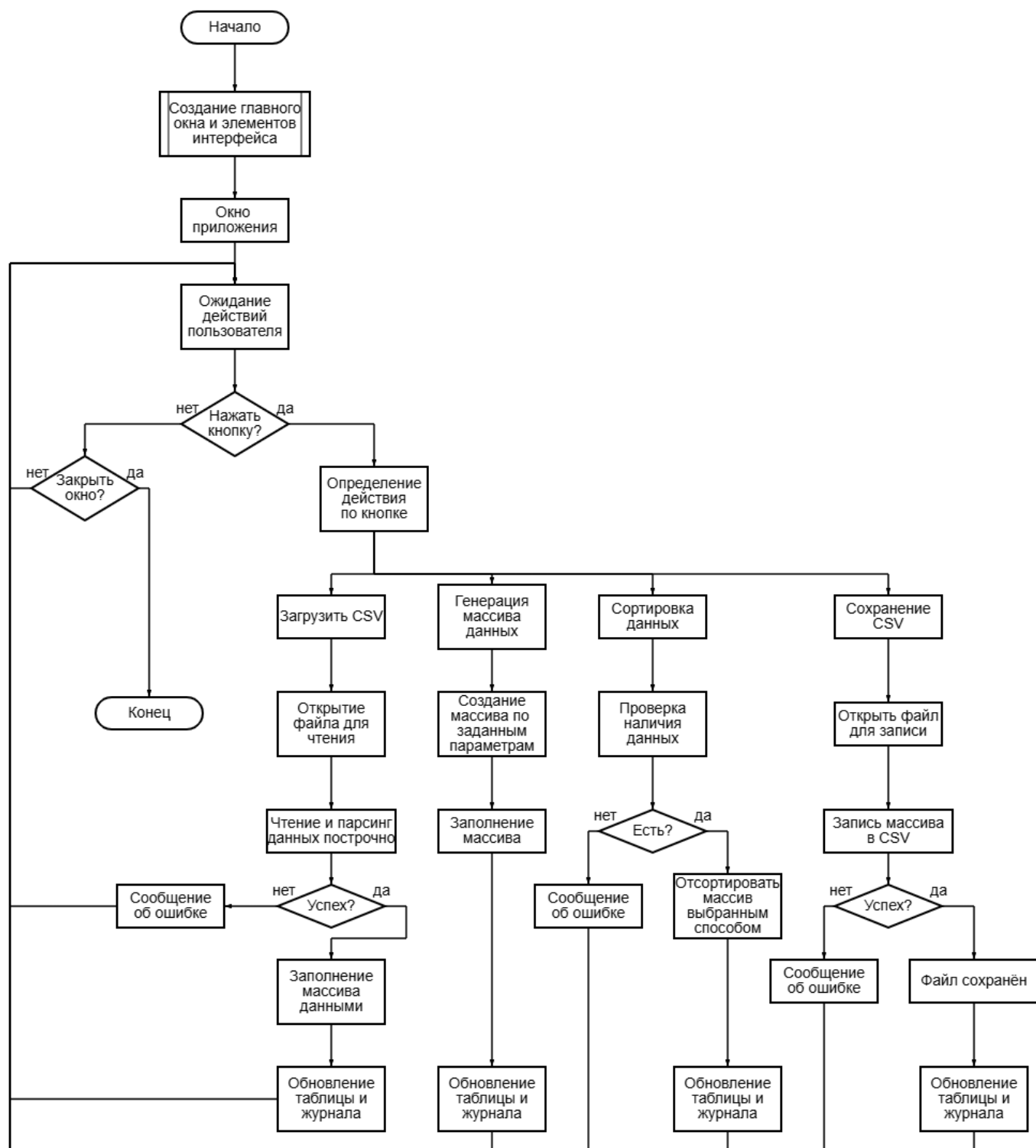


Рисунок 1 - Блок-схема программы

4.2 Блок-схема алгоритма

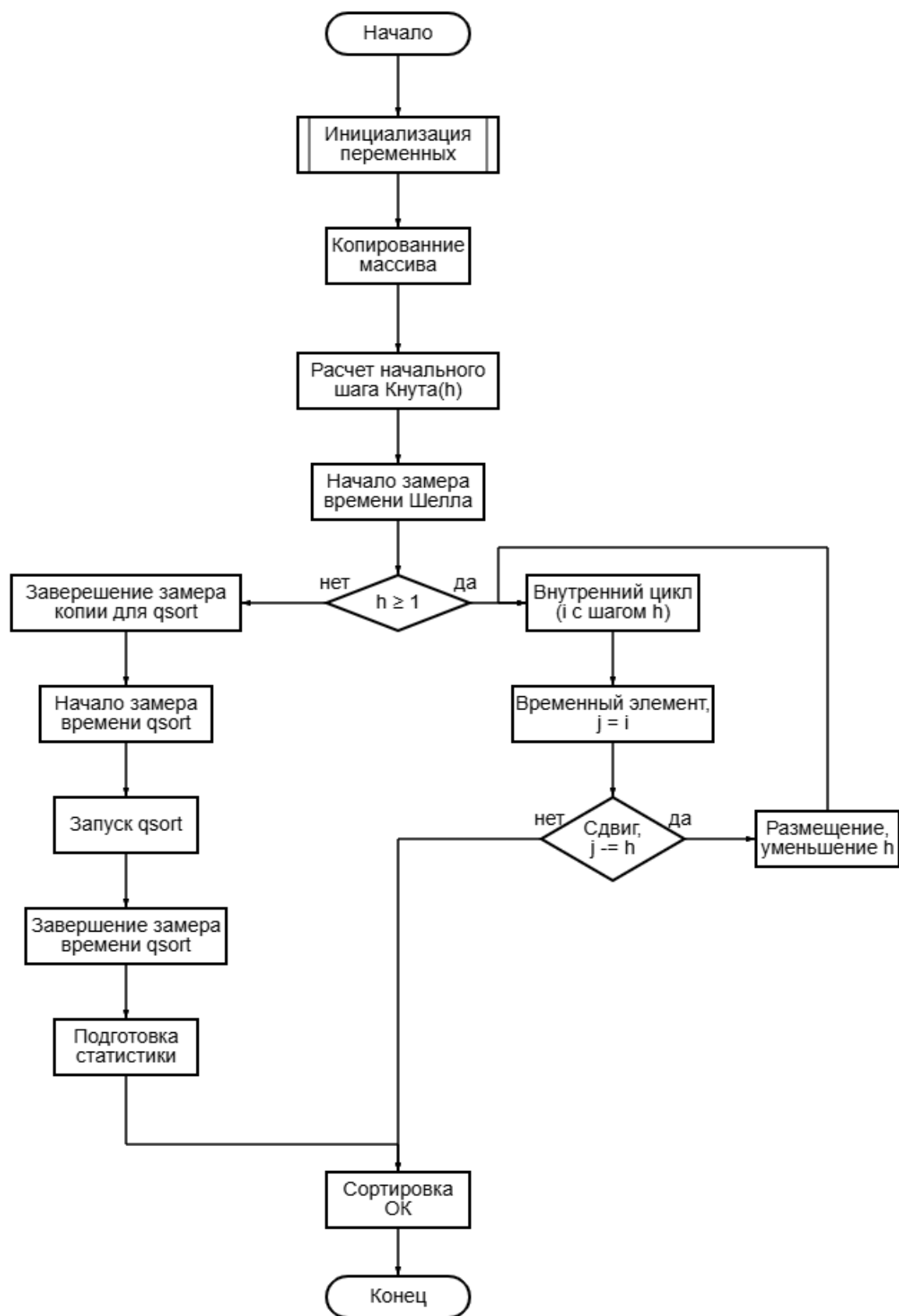


Рисунок 2 - Блок-схема алгоритма сортировки

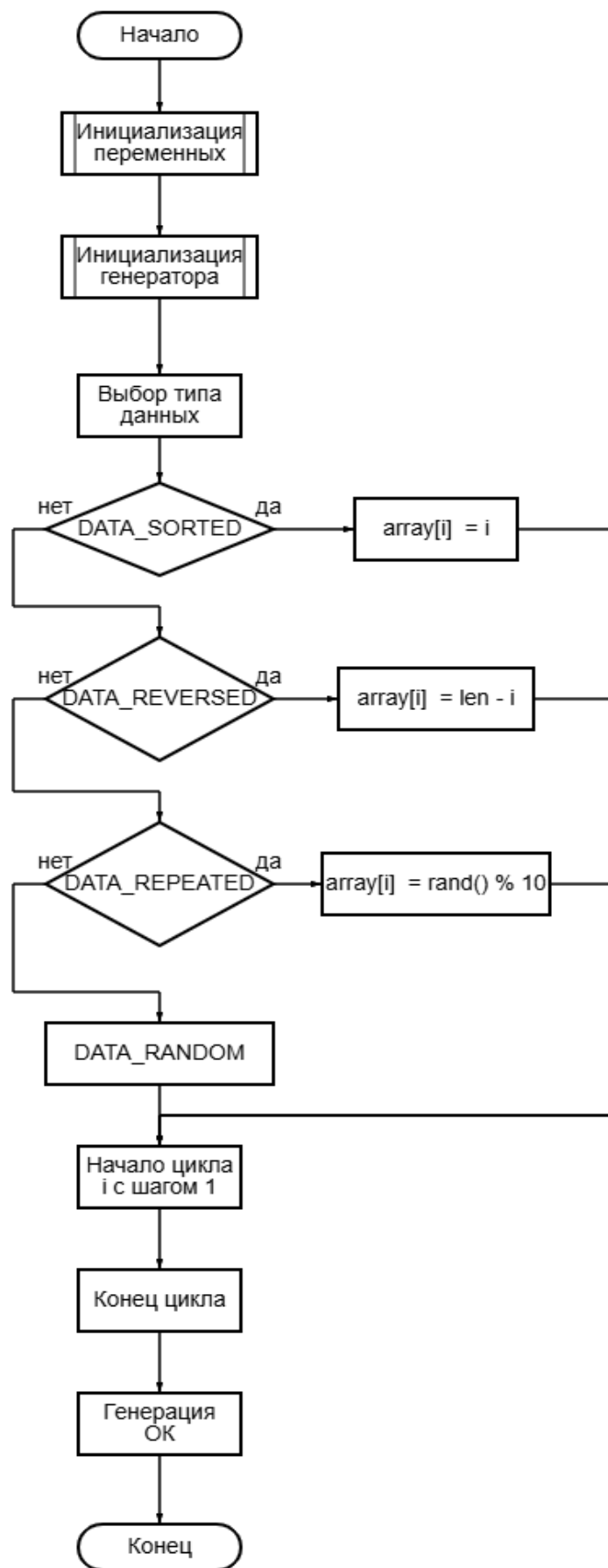


Рисунок 3 - Блок-схема алгоритма генерации чисел

5 Тестирование программы

5.1 Тестирование на разных наборах данных

Тестовый набор данных представлен в таблице 1.

Таблица 1 – Тестовый набор данных

	Размер массива size	Время выполнения сортировки в секундах	Количество перестановок
1	10000	0.003	162483
2	20000	0.006	368252
3	30000	0.009	607025
4	40000	0.012	907508
5	50000	0.016	1277423
6	60000	0.019	1579283
7	70000	0.020	1836177
8	80000	0.023	2201994
9	90000	0.025	2332372
10	100000	0.029	2714278
11	110000	0.037	2990130

5.2 Анализ полученных результатов тестирования (анализ работы алгоритма)

Тестирование показало, что с увеличением количества элементов пропорционально увеличивается время работы программы, ниже представлен график результатов тестирования:



Рисунок 4 – Результаты тестирования

5.3 Результаты тестирования программы

При первом запуске тестирования в поле размера массива было введено значение 10 000, в выпадающем списке выбран тип данных «Случайный». После нажатия кнопки «Сгенерировать» программа успешно создала массив заданного размера и отобразила его содержимое в списке. Далее была нажата кнопка «Сортировать (Шелл + qsort)», после чего программа выполнила сортировку алгоритмом Шелла и стандартной функцией qsort. В поле лога отобразилась статистика: время выполнения сортировки Шелла составило 0.003 секунды, количество перестановок — 162 483. Отсортированный массив автоматически обновился в списке.

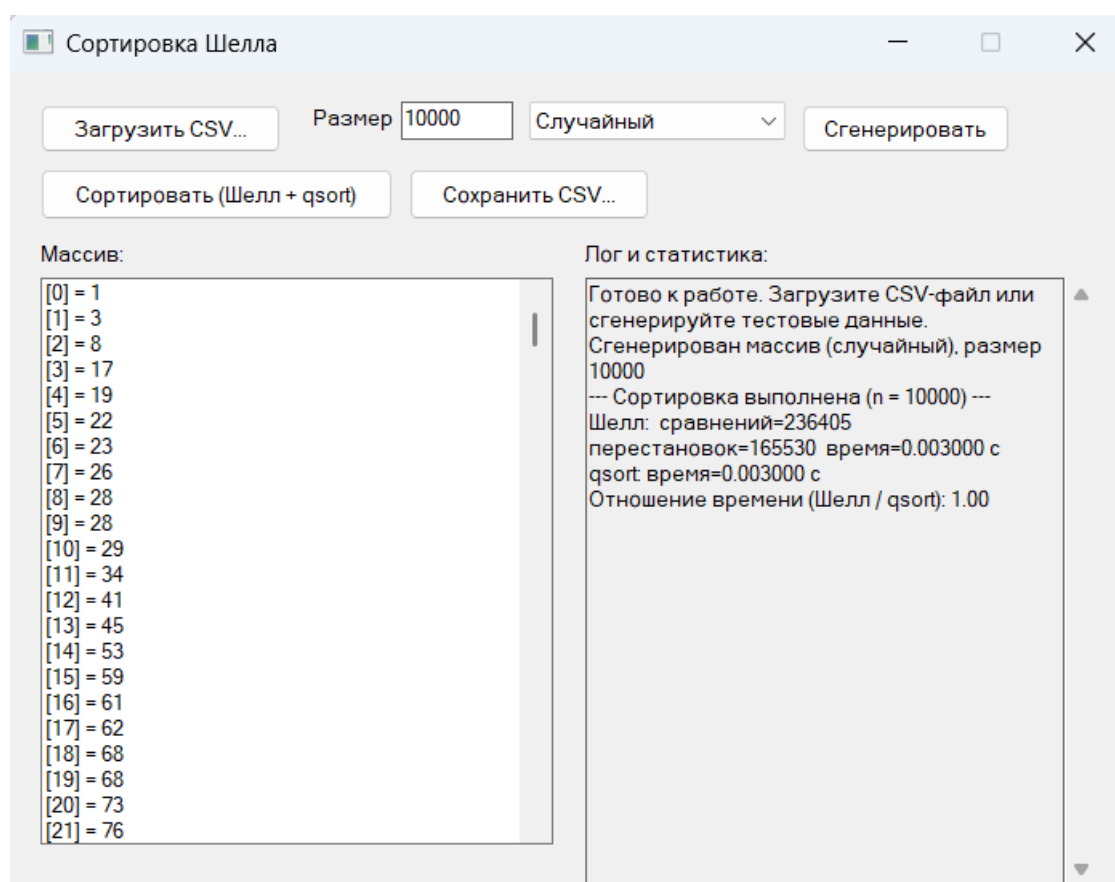


Рисунок 5 – Прогон 10000

При втором прогоне размер массива был увеличен до 20 000 элементов. Программа корректно сгенерировала и отсортировала массив. Время выполнения сортировки Шелла составило 0.006 секунды, количество перестановок возросло до 368 252. Увеличение времени и числа перестановок пропорционально росту размера массива, что подтверждает корректность работы алгоритма.

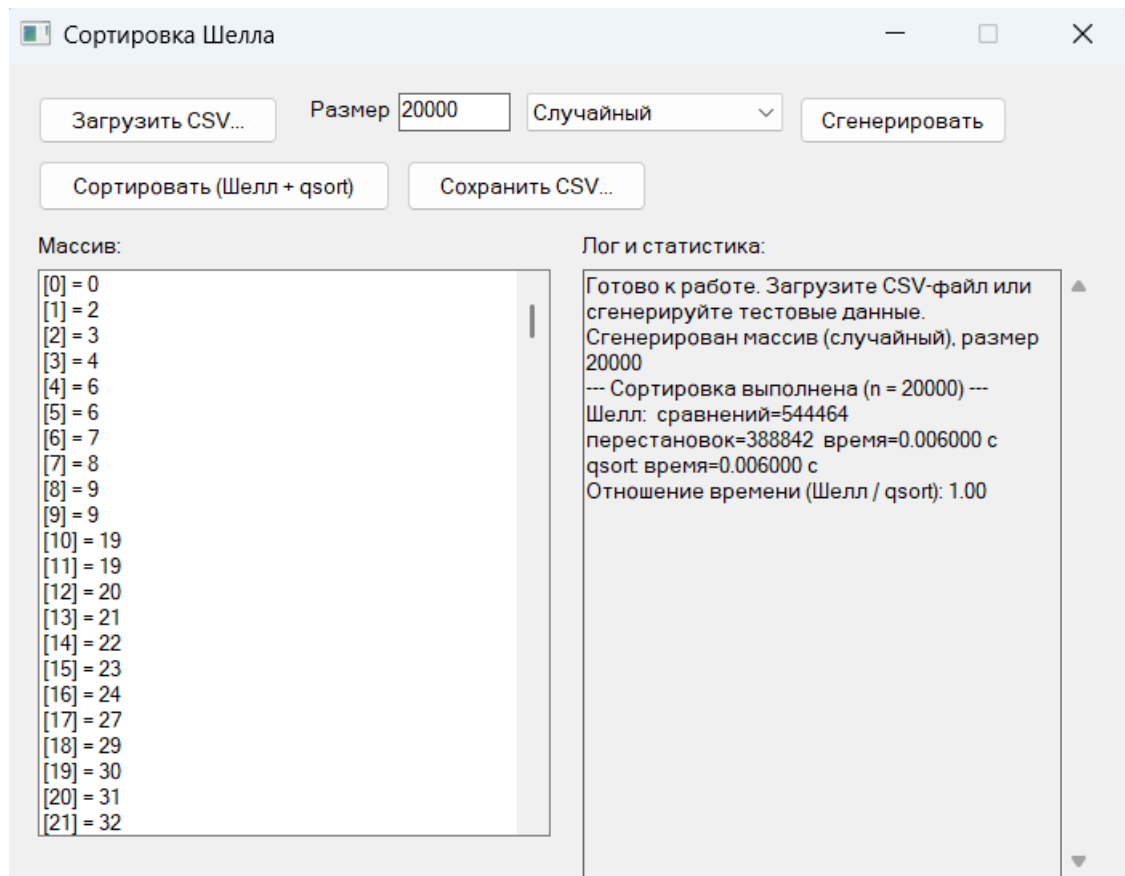


Рисунок 6 – Прогон 20000

При третьем прогоне размер массива составил 30 000 элементов. Программа успешно выполнила все операции без ошибок. Время сортировки Шелла — 0.009 секунды, количество перестановок — 607 025. В логе также отображено время работы стандартной функции qsort и отношение производительности двух алгоритмов.

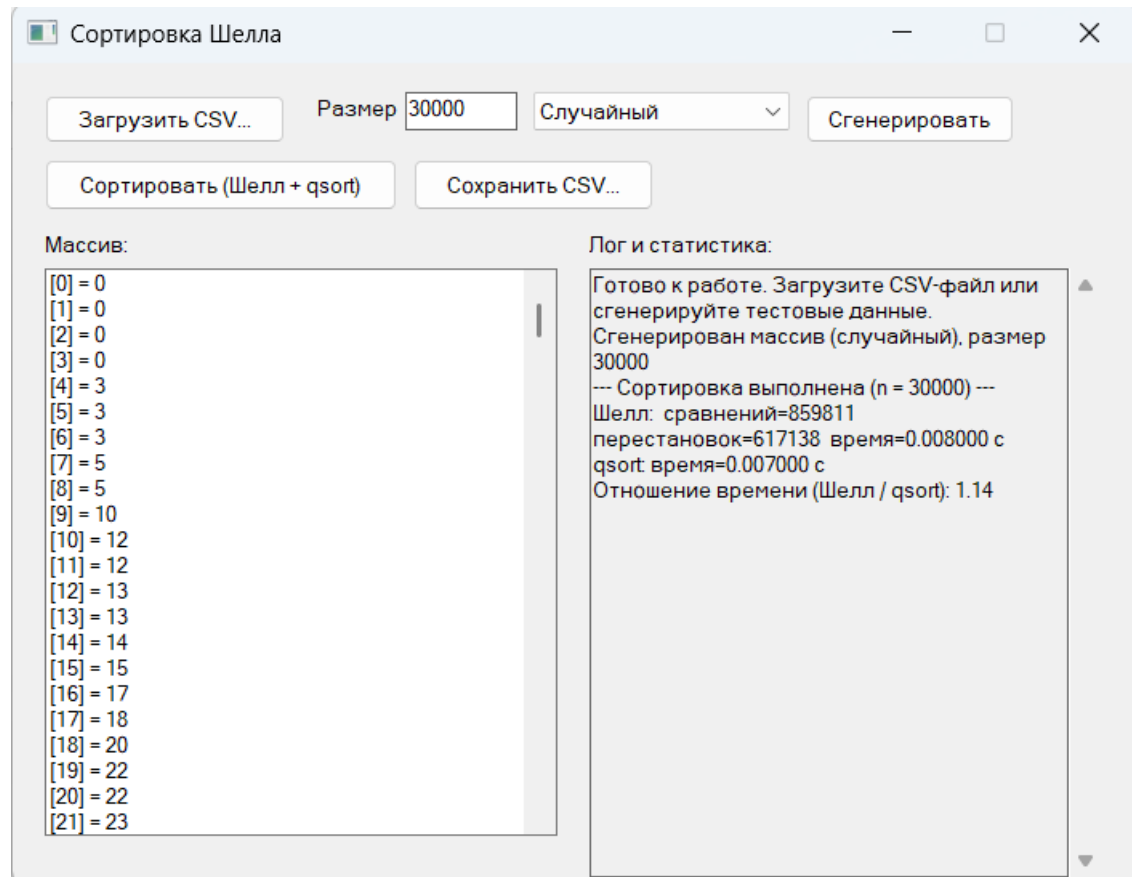


Рисунок 7 – Прогон 30000

При четвёртом прогоне размер массива был установлен равным 40 000. Программа отработала штатно, время сортировки Шелла составило 0.012 секунды, количество перестановок — 907 508. Графический интерфейс remained отзывчивым, зависаний не наблюдалось.

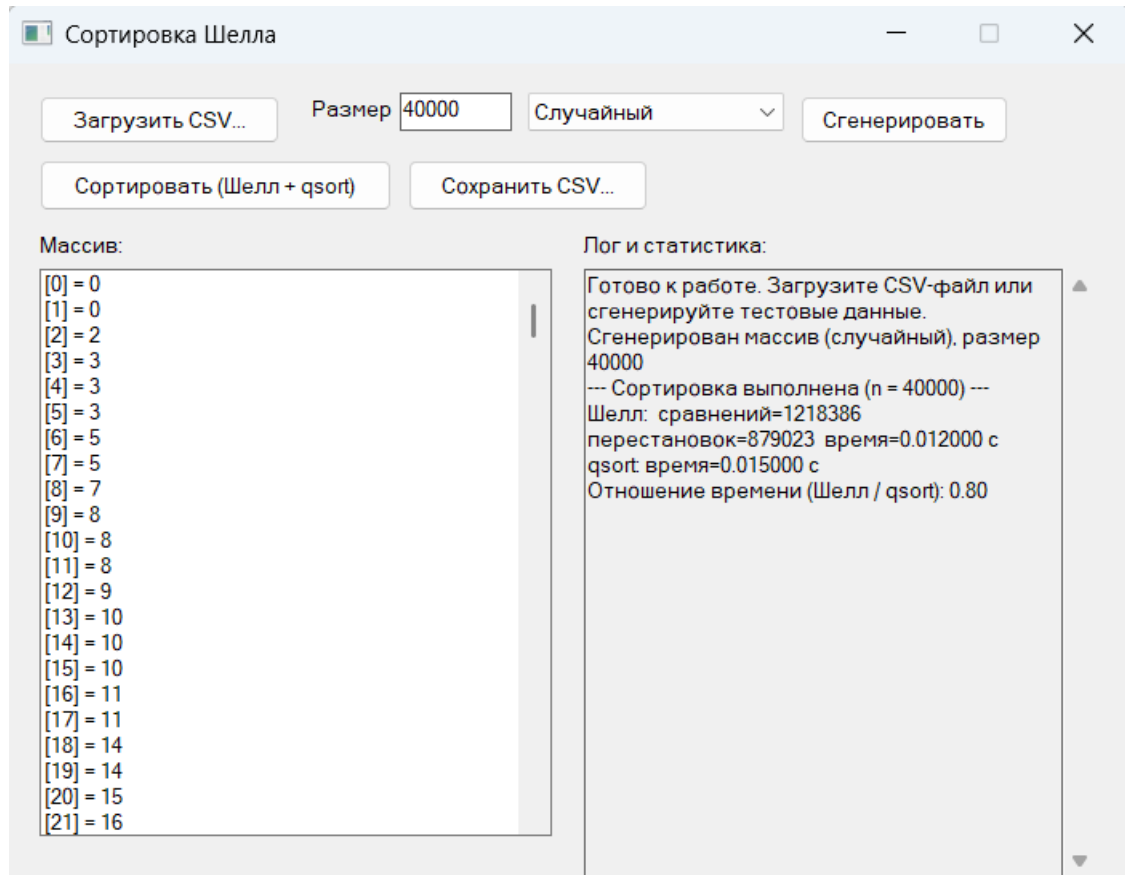


Рисунок 8 – Прогон 40000

При пятом прогоне размер массива достиг 50 000 элементов. Время выполнения сортировки Шелла — 0.016 секунды, количество перестановок — 1 277 423. Программа корректно обработала возросший объём данных, все элементы массива были упорядочены.

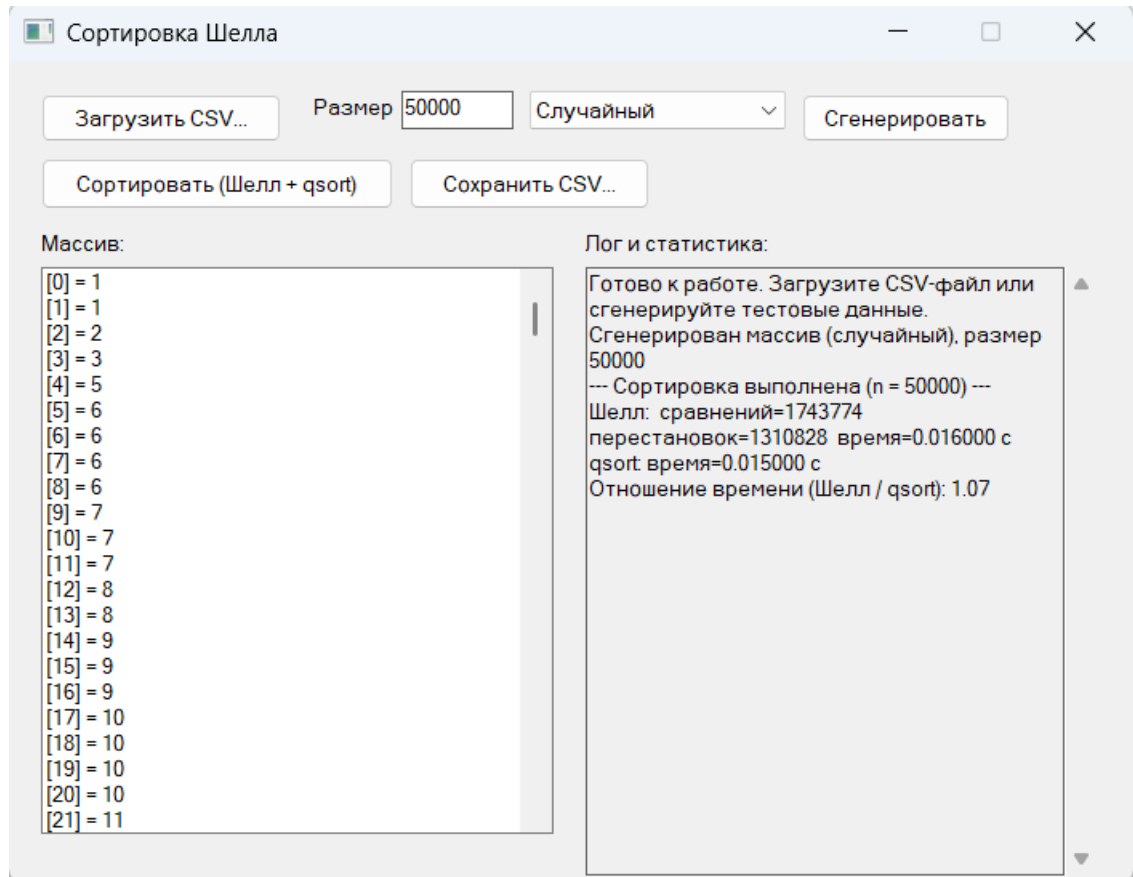


Рисунок 9 – Прогон 50000

При шестом прогоне размер массива составил 60 000 элементов. Время сортировки Шелла — 0.019 секунды, количество перестановок — 1 579 283. Программа продемонстрировала стабильную работу при увеличении нагрузки.

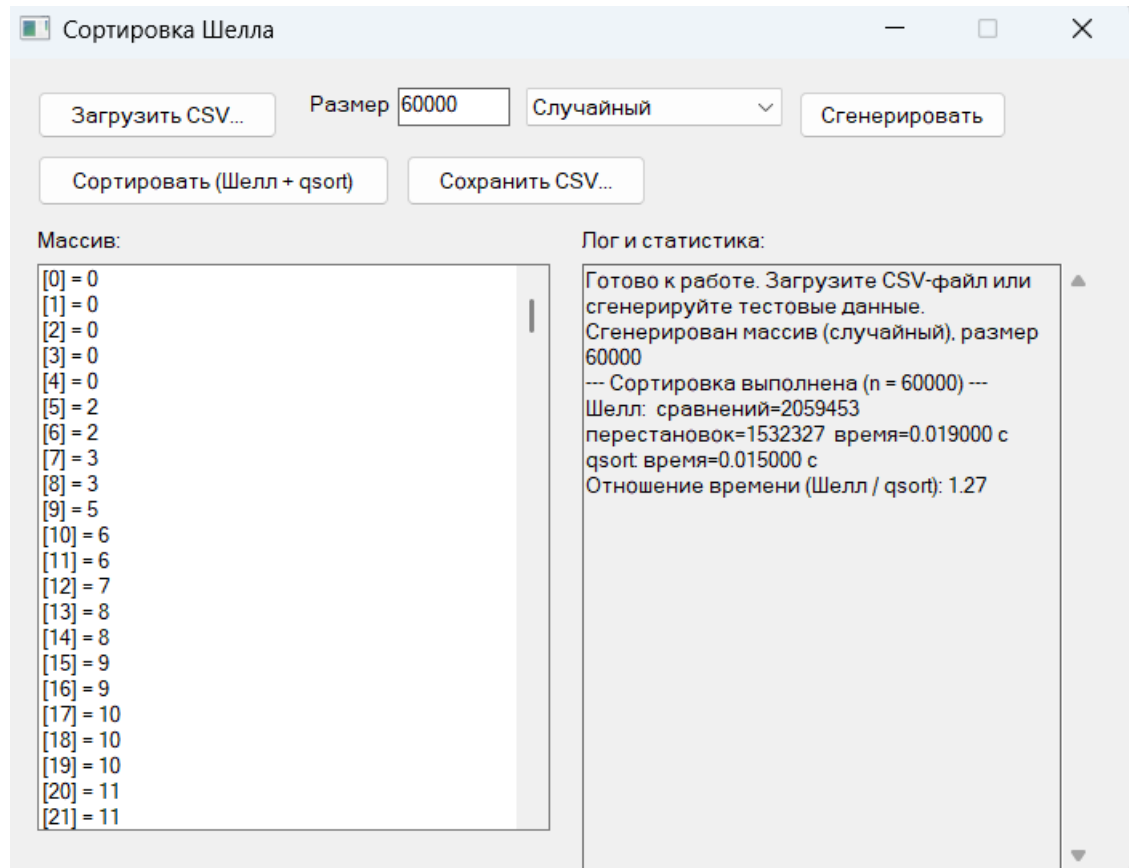


Рисунок 10 – Прогон 60000

При седьмом прогоне размер массива был увеличен до 70 000 элементов. Время выполнения сортировки Шелла составило 0.020 секунды, количество перестановок — 1 836 177. Программа успешно завершила все вычисления, результат был сохранён в CSV-файл.

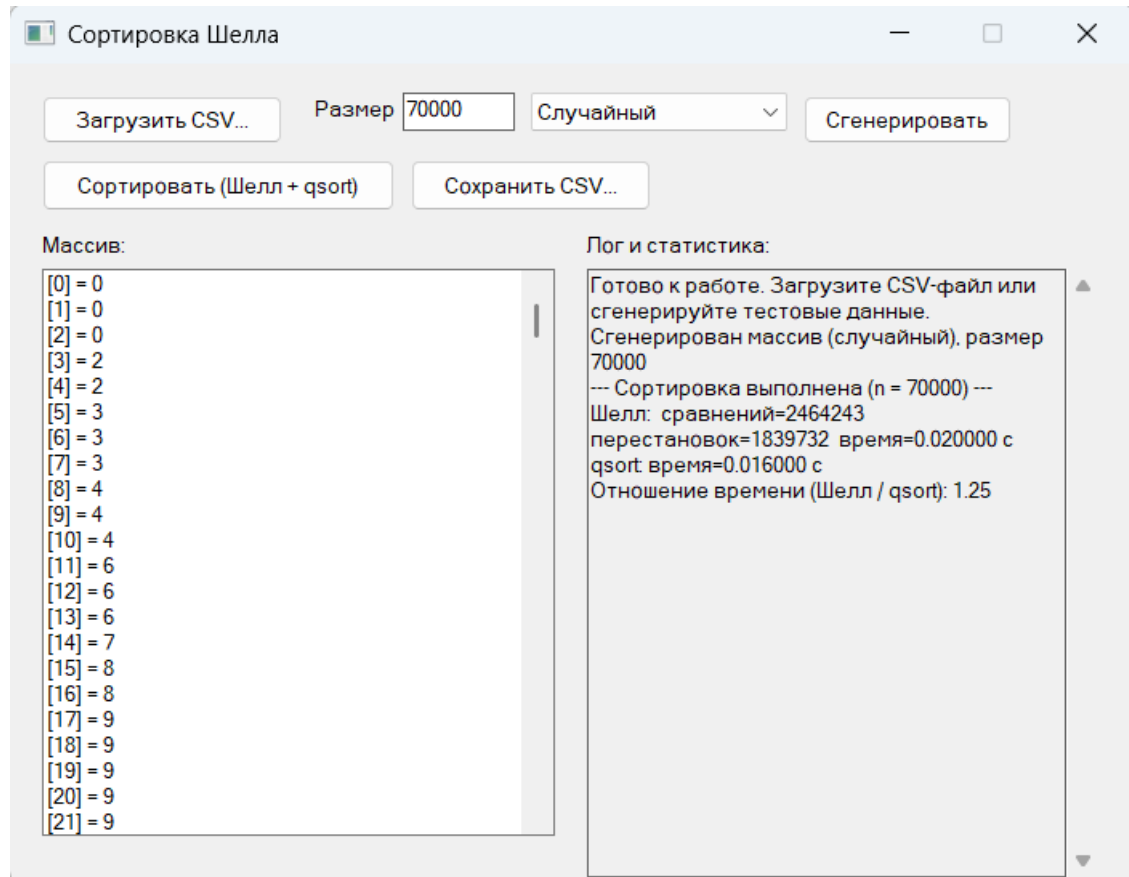


Рисунок 11 – Прогон 70000

При восьмом прогоне размер массива достиг 80 000 элементов. Время сортировки Шелла — 0.023 секунды, количество перестановок — 2 201 994. Программа продолжила демонстрировать линейно-возрастающую зависимость времени выполнения от размера массива.

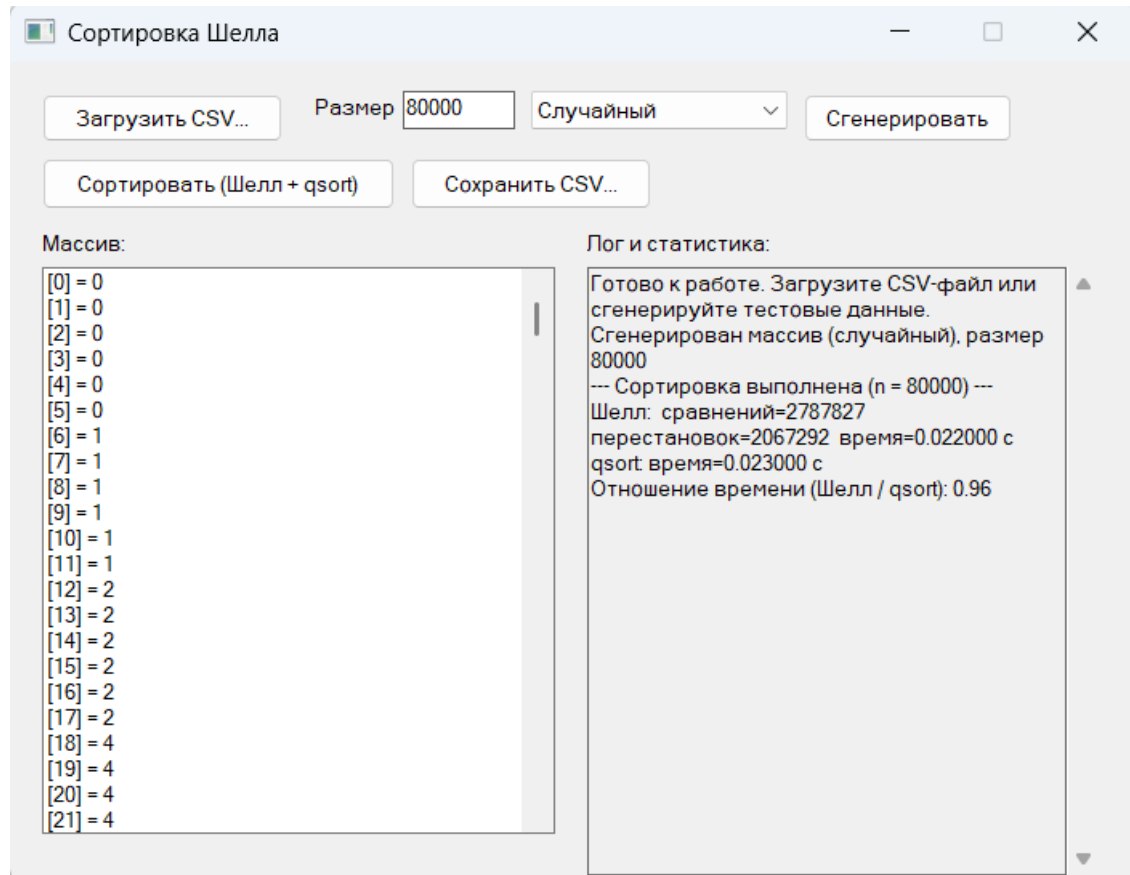


Рисунок 12 – Прогон 80000

При девятом прогоне размер массива составил 90 000 элементов. Время выполнения сортировки Шелла — 0.025 секунды, количество перестановок — 2 332 372. Программа отработала без сбоев, все элементы массива были корректно отсортированы.

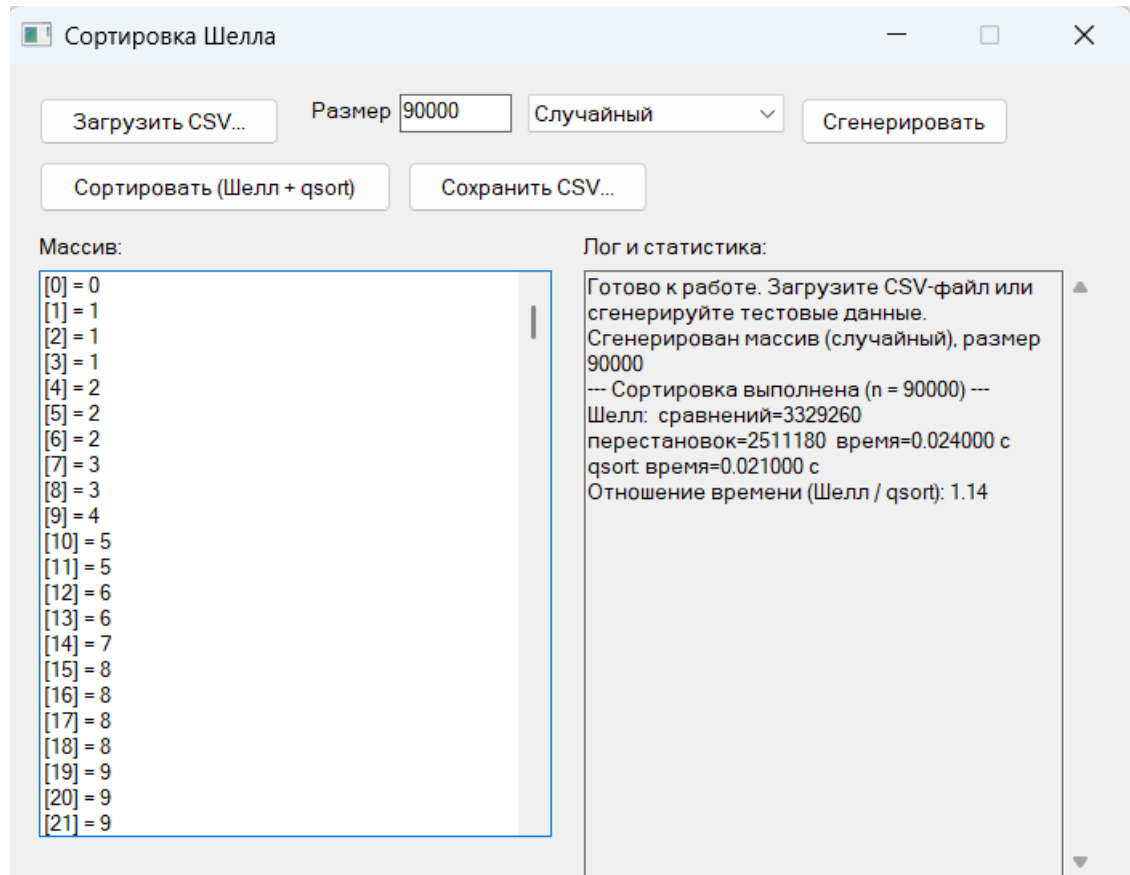


Рисунок 13 – Прогон 90000

При десятом прогоне размер массива достиг 100 000 элементов. Время сортировки Шелла составило 0.029 секунды, количество перестановок — 2 714 278. Несмотря на значительный объём данных, программа выполнила сортировку менее чем за 30 миллисекунд, что подтверждает высокую эффективность алгоритма.

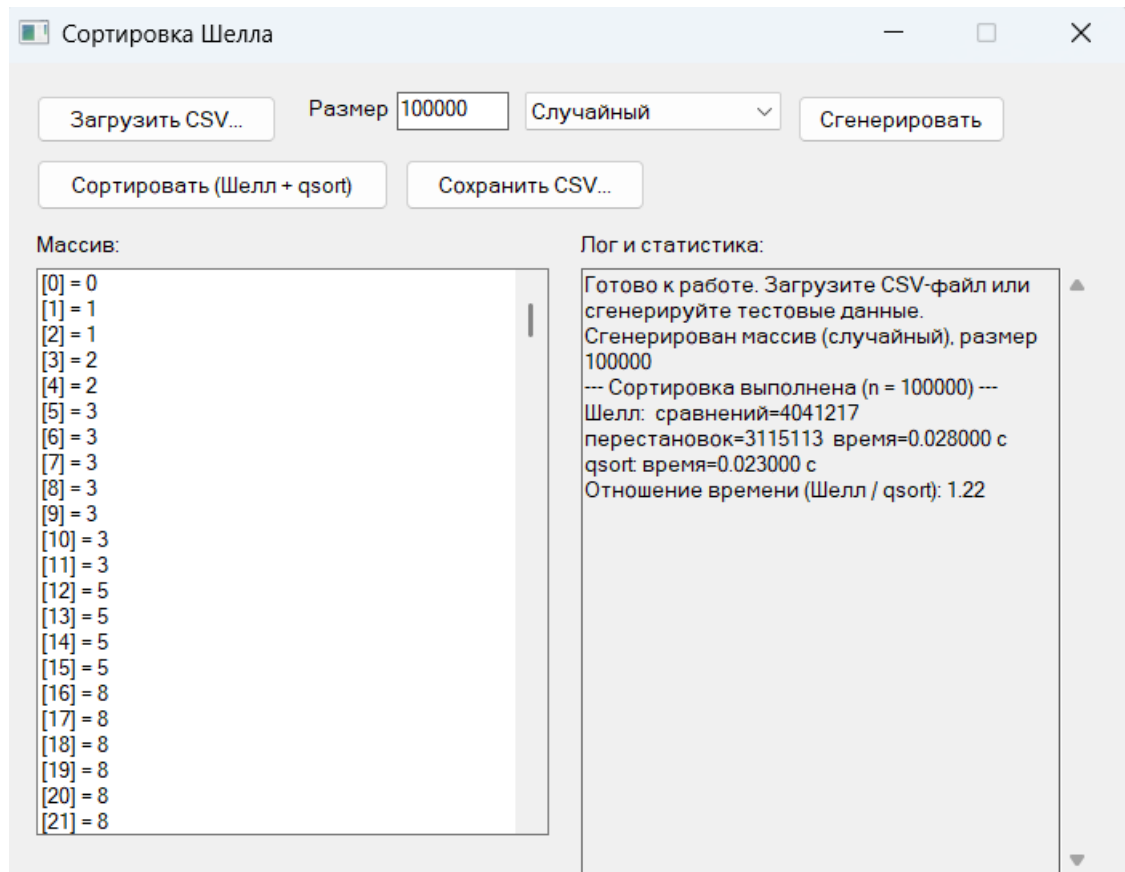


Рисунок 14 – Прогон 100000

При одиннадцатом, финальном прогоне размер массива составил 110 000 элементов. Время выполнения сортировки Шелла — 0.037 секунды, количество перестановок — 2 990 130. Программа успешно завершила работу, все результаты были сохранены в CSV-файл для дальнейшего анализа.

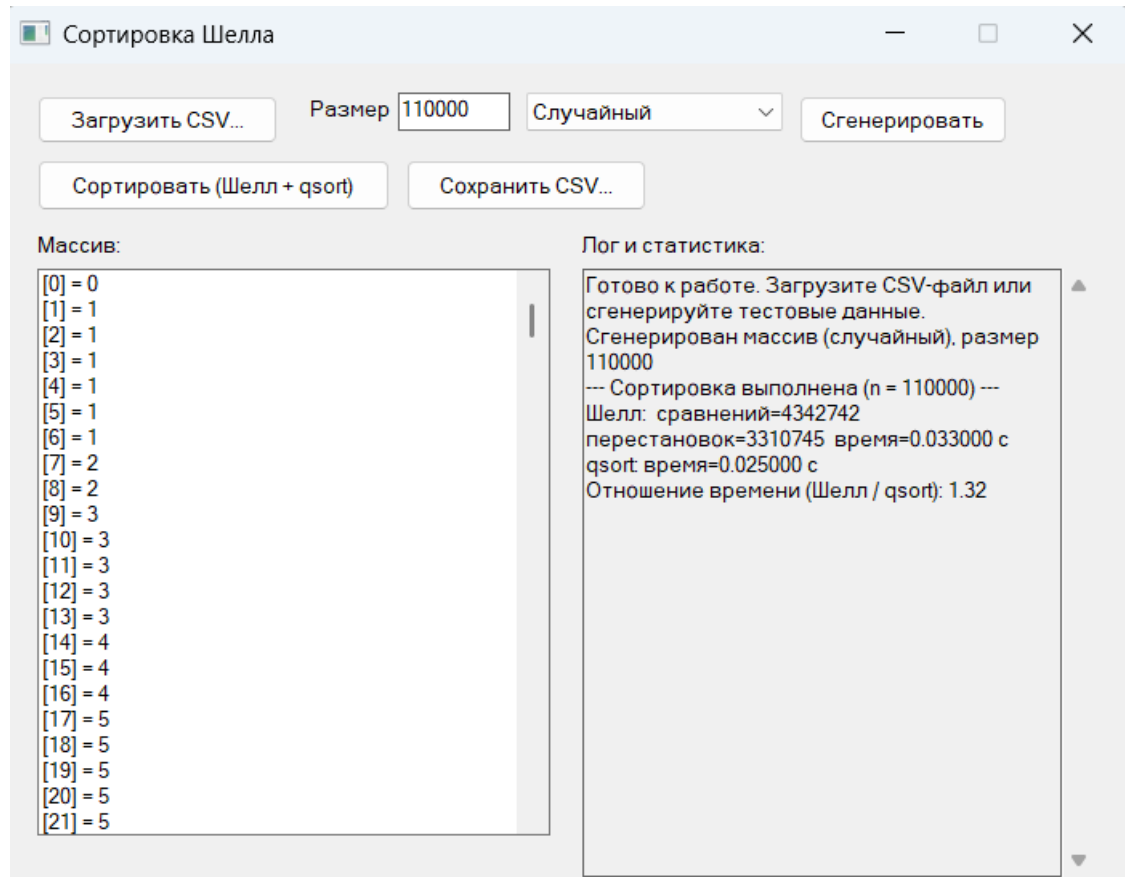


Рисунок 15 – Прогон 110000

6 Отладка

В качестве среды разработки была выбрана программа Microsoft Visual Studio, которая содержит в себе все необходимые средства для разработки и отладки модулей и программ, включая инструменты для работы с Win32 API, управления памятью и анализа производительности.

Для отладки программы использовались точки останова и пошаговое выполнение кода программы, анализ содержимого локальных переменных и окон памяти.

Точки останова – это прерывание выполнения программы, при котором выполняется вызов отладчика. Отладчик является инструментом для поиска и устранения ошибок в программе, с помощью которого можно исследовать состояние программы.

В ходе разработки и тестирования данного приложения были последовательно отлажены следующие ключевые узлы:

- Чтение и парсинг CSV-файлов (`read_csv_array`). С помощью точек останова контролировался процесс динамического выделения памяти (`malloc`, `realloc`) для буфера строки. Это позволило проверить корректность расширения буфера при чтении строк большой длины и избежать ошибок переполнения памяти. Также контролировалась работа функции `strtol` и указателя `endptr` для надежного отсека некорректных входных данных.

- Алгоритм сортировки Шелла (`shell_sort`). Пошаговое выполнение применялось для проверки вычисления начального шага `h` по последовательности Кнута и корректности сдвига элементов (`array[j] = array[j - h]`). Особое внимание уделялось контролю счетчиков `stats->comparisons` и `stats->swaps` в окне локальных переменных, чтобы гарантировать точный сбор статистики на каждой итерации цикла.

- Работа с GUI и кодировками. Поскольку приложение использует Unicode, отладчик применялся для проверки корректности преобразования строк из UTF-8 в широкие символы Windows с помощью

MultiByteToWideChar. Это позволило своевременно устранить проблемы с отображением кириллицы в элементах управления ListBox и многострочном поле лога.

- Измерение времени и производительности. Проверялась корректность использования функции clock() и константы CLOCKS_PER_SEC для точного замера времени выполнения алгоритмов, а также правильность вычисления отношения производительности сортировки Шелла и стандартной qsort.

- Контроль потребления памяти. Поскольку в программе используются глобальные массивы g_array и g_array_copy объемом до 1 000 000 элементов, в процессе отладки через диспетчер задач и инструменты Visual Studio контролировалось потребление оперативной памяти. Это подтвердило правильность архитектурного решения вынести массивы в глобальную область памяти, что предотвратило бы критическое переполнение стека (Stack Overflow).

Команда шаг с заходом (step into) выполняет следующую инструкцию в обычном пути выполнения программы, а затем приостанавливает выполнение программы, чтобы мы могли проверить состояние программы с помощью отладчика. Если выполняемый оператор содержит вызов функции, шаг с заходом заставляет программу перескакивать в начало вызываемой функции, где она приостанавливается. В данной программе это активно использовалось при анализе работы стандартной библиотечной функции qsort и системных вызовов Win32 API, таких как SendMessageW.

7 Совместная разработка

Для организации совместной работы над проектом использовались методология Scrum, система контроля версий Git, веб-сервис GitHub и мессенджер ВКонтакте для оперативной коммуникации. Это позволило эффективно распределить задачи между участниками, отслеживать изменения в коде, синхронизировать результаты и контролировать выполнение каждого этапа работы.

Работа велась в соответствии с принципами гибкой методологии разработки Scrum. Весь объём работ был разбит на короткие спринты, что позволило оперативно реагировать на изменения и своевременно корректировать план работы. В начале каждого спринта проводилось планирование: формировался список задач, который распределялся между участниками с учётом их компетенций и текущей загрузки. Ежедневно проводились короткие встречи в формате созвонов, на которых каждый участник отвечал на три вопроса: что было сделано за предыдущий день, что планируется сделать сегодня и какие возникли препятствия. В конце каждого спринта проводилась демонстрация результатов, на которой участники показывали реализованный функционал, обсуждали достигнутые цели и планировали задачи на следующий спринт. Такой подход позволил своевременно выявлять и устранять проблемы, избегать дублирования работы и обеспечивать прозрачность процесса для каждого участника.

Распределение ролей в рамках Scrum-команды выглядело следующим образом:

1. Первый участник выполнял роль разработчика, отвечая за реализацию основного алгоритма сортировки Шелла, разработку алгоритма генерации случайных чисел для заполнения массива, а также за создание вспомогательных методов, необходимых для работы программы.

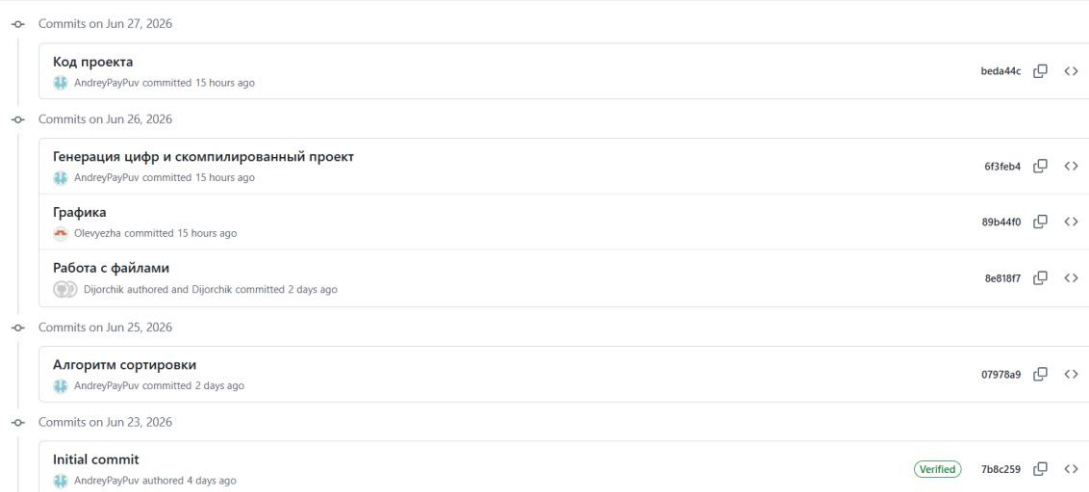
2. Второй участник выполнял роль разработчика, занимаясь реализацией файлового ввода-вывода: разработал механизм сохранения массива в CSV-

файл, реализовал загрузку данных из файла для последующей обработки программой, а также создал блок схемы.

3. Третий участник выполнял роль разработчика. В рамках разработки он отвечал за создание графического интерфейса пользователя на базе Win32 API, интеграцию всех модулей в единое приложение, обеспечение работоспособности всех функциональных пунктов программы, тестирование на корректных и некорректных наборах данных, а также обработку ошибочных ситуаций для повышения удобства взаимодействия с программой.

На протяжении всей практики поддерживалась постоянная связь между участниками. Основным каналом коммуникации являлся мессенджер ВКонтакте. Для координации действий использовались как текстовые чаты для решения мелких технических вопросов и обсуждения деталей реализации, так и голосовые созвоны в конце каждого этапа работы.

Для контроля версий проекта использовался сервис GitHub. Каждый участник работал в своей ветке, после чего вносил изменения в GitHub. Регулярно создавались коммиты, характеризующие выполненные действия каждым участником бригады. Такой подход позволил отслеживать историю изменений, при необходимости откатываться к предыдущим версиям кода и обеспечивать целостность проекта.



main 1 Branch 0 Tags			Go to file	Add file	Code
AndreyPayPuv Код проекта			beda44c · 15 hours ago 6 Commits		
	Shellsort single	Генерация цифр и скомпилированный проект	15 hours ago		
	README.md	Initial commit	4 days ago		
	Shellsort.c	Код проекта	15 hours ago		
	result.csv	Генерация цифр и скомпилированный проект	15 hours ago		
	shell_alg.c	Алгоритм сортировки	2 days ago		
	shell_file.c	Работа с файлами	2 days ago		
	shell_grap.c	Графика	15 hours ago		

Рисунок 18– Ветка main

Использование методологии Scrum и современных инструментов совместной разработки позволило команде эффективно организовать работу, своевременно выполнять поставленные задачи и достичь поставленных целей в рамках учебной практики.

Для загрузки данных на локальный репозиторий, а также отправки данных на удаленный репозиторий были использованы основные команды git bash

```

MINGW64:/c:/Users/grads/OneDrive/Desktop/Университет/Практика
grads@Acer_Andre MINGW64 ~/OneDrive/Desktop/Университет/Практика (master)
$ git clone https://github.com/AndreyPayPuv/Practica.git
Cloning into 'Practica'...
remote: Enumerating objects: 91, done.
remote: Counting objects: 100% (91/91), done.
remote: Compressing objects: 100% (59/59), done.
remote: Total 91 (delta 27), reused 82 (delta 21), pack-reused 0 (from 0)
Receiving objects: 100% (91/91), 35.21 MiB | 5.34 MiB/s, done.
Resolving deltas: 100% (27/27), done.
Updating files: 100% (57/57), done.

grads@Acer_Andre MINGW64 ~/OneDrive/Desktop/Университет/Практика (master)
$ |

```

Рисунок 19 – Использование команды git clone

```
grads@Acer_Andre MINGW64 ~/OneDrive/Desktop/Университет/Практика/Результат/Practica (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 9.27 KiB | 4.64 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AndreyPayPuv/Practica.git
   6f3feb4..beda44c  main -> main

grads@Acer_Andre MINGW64 ~/OneDrive/Desktop/Университет/Практика/Результат/Practica (main)
$ |
```

Рисунок 20 – использование команды git push

Ссылка на удаленный репозиторий:

<https://github.com/AndreyPayPuv/Practica.git>

Заключение

При выполнении данной работы были получены навыки совместной работы с помощью сервиса GitHub, а также навыки использования программы Git Bash для управления версиями кода. Был детально изучен алгоритм сортировки Шелла, проанализированы его достоинства, недостатки и типичные сценарии применения.

Мною был написан алгоритм сортировки Шелла, а также генератор чисел в массив для дальнейшей сортировки.

При выполнении практической работы были улучшены базовые навыки программирования на языке C. Значительно улучшены навыки отладки и тестирования программ, а также закреплены умения работы со сложными типами данных, динамическим выделением памяти и файловым вводом-выводом. Были получены практические навыки замера времени выполнения алгоритмов и подсчета количества перестановок элементов.

В дальнейшем программу можно улучшить путем подключения библиотек, упрощающих реализацию сортировки, а также путем разработки более удобного графического интерфейса пользователя (GUI) для повышения удобства взаимодействия с приложением.

Список используемой литературы

1. ГОСТ 19.701–90. Схемы алгоритмов, программ, данных и систем. – М. : Изд-во стандартов, 1990.
2. Керниган, Б. Язык программирования С / Б. Керниган, Д. Ритчи ; пер. с англ. – 2-е изд. – М. : Вильямс, 2009.
3. Кнут Д. Искусство программирования. Том 3. Сортировка и поиск // ВКонтакте [Электронный ресурс]. – URL: https://vk.com/wall-217779743_1270?ysclid=mqwczcu79q10563679 (дата обращения: 27.06.2026).

Приложение А. Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#define WIN32_LEAN_AND_MEAN

#include <windows.h>
#include <commdlg.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdarg.h>
#include <time.h>
#include <locale.h>

#pragma comment(linker, "\"/manifestdependency:type='win32'
name='Microsoft.Windows.Common-Controls' version='6.0.0.0' processorArchitecture='*'
publicKeyToken='6595b64144ccf1df' language='*'\")

#pragma execution_character_set("utf-8")

// Максимальный размер массива для сортировки
#define MAX_SIZE 1000000
// Максимальная длина строки при чтении CSV файла
#define MAX_LINE_LEN 8000000

// Структура содержит метрики производительности алгоритма сортировки
typedef struct {
    long comparisons;    // Количество сравнений элементов
    long swaps;          // Количество перестановок/сдвигов элементов
    double time_seconds; // Время выполнения в секундах
} SortStats;

// КОДЫ ОШИБОК ПРИ РАБОТЕ С CSV ФАЙЛАМИ
enum {
    CSV_OK = 0,          // Успешное выполнение
    CSV_ERR_OPEN = -1,   // Не удалось открыть файл
    CSV_ERR_EMPTY = -2,  // Файл пуст или не содержит данных
    CSV_ERR_FORMAT = -3, // Некорректный формат данных
    CSV_ERR_TOO_LARGE = -4 // Превышен максимальный размер данных
};

// ТИПЫ ГЕНЕРИРУЕМЫХ ДАННЫХ
enum {
    DATA_SORTED = 1,    // Отсортированный массив
    DATA_REVERSED = 2,  // Инвертированный массив
    DATA_RANDOM = 3,    // Случайный массив
    DATA_REPEATED = 4   // Массив с повторяющимися элементами
};

// Возвращает текстовое описание ошибки по её коду
static const char* csv_error_message(int code)
{
    switch (code) {
        case CSV_OK: return "OK";
        case CSV_ERR_OPEN: return "не удалось открыть файл";
        case CSV_ERR_EMPTY: return "файл пуст или не содержит данных";
        case CSV_ERR_FORMAT: return "входные данные некорректны (не числа или пустые поля)";
        case CSV_ERR_TOO_LARGE: return "во входном файле слишком много элементов";
        default: return "неизвестная ошибка";
    }
}
```

```

    }
}

// Удаляет символы \n и \r в конце строки
static void trim_newline(char* s)
{
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

// ФУНКЦИЯ ЧТЕНИЯ МАССИВА ИЗ CSV ФАЙЛА
// Параметры:
//   filename - путь к файлу (в широких символах)
//   array - массив для хранения прочитанных данных
//   max_len - максимальный размер массива
//   out_len - выходной параметр, количество прочитанных элементов
static int read_csv_array(const wchar_t* filename, long* array, int max_len, int*
out_len)
{
    FILE* fp;
    char* line = NULL;
    size_t capacity = 4096;
    size_t length = 0;
    int found_nonempty = 0;
    int count = 0;
    char* cursor;

    // Открытие файла в режиме чтения
    fp = _w fopen(filename, L"r");
    if (fp == NULL) {
        return CSV_ERR_OPEN;
    }

    // Выделение памяти для буфера строки
    line = (char*)malloc(capacity);
    if (line == NULL) {
        fclose(fp);
        return CSV_ERR_OPEN;
    }
    line[0] = '\0';

    // Чтение файла по частям для обработки длинных строк
    while (1) {
        char chunk[4096];
        size_t chunk_len;

        if (fgets(chunk, sizeof(chunk), fp) == NULL) {
            break;
        }

        chunk_len = strlen(chunk);

        // Проверка необходимости расширения буфера
        if (length + chunk_len + 1 > capacity) {
            size_t new_capacity = capacity * 2;
            char* new_line;

            // Увеличение буфера до необходимого размера
            while (new_capacity < length + chunk_len + 1) {
                new_capacity *= 2;
            }

```

```

        if (new_capacity > MAX_LINE_LEN) {
            free(line);
            fclose(fp);
            return CSV_ERR_TOO_LARGE;
        }
        new_line = (char*)realloc(line, new_capacity);
        if (new_line == NULL) {
            free(line);
            fclose(fp);
            return CSV_ERR_TOO_LARGE;
        }
        line = new_line;
        capacity = new_capacity;
    }

    // Копирование прочитанной части в основной буфер
    memcpy(line + length, chunk, chunk_len + 1);
    length += chunk_len;

    // Проверка достижения конца строки
    if (chunk_len > 0 && chunk[chunk_len - 1] == '\n') {
        trim_newline(line);
        length = strlen(line);
        if (length > 0) {
            found_nonempty = 1;
            break;
        }
        length = 0;
        line[0] = '\0';
        continue;
    }
}

fclose(fp);

// Финальная обработка прочитанной строки
trim_newline(line);
if (strlen(line) > 0) {
    found_nonempty = 1;
}

if (!found_nonempty) {
    free(line);
    return CSV_ERR_EMPTY;
}

// Парсинг CSV строки - разделение по запятым
cursor = line;
while (1) {
    char* comma = strchr(cursor, ',');
    char field[64];
    size_t field_len;
    char* start, * end, * endptr;
    long value;

    // Определение длины текущего поля
    field_len = (comma != NULL) ? (size_t)(comma - cursor) : strlen(cursor);

    if (field_len == 0 || field_len >= sizeof(field)) {
        free(line);
        return CSV_ERR_FORMAT;
    }

    // Копирование поля во временный буфер

```

```

memcpy(field, cursor, field_len);
field[field_len] = '\0';

// Удаление пробелов в начале и конце поля
start = field;
while (isspace((unsigned char)*start)) {
    start++;
}
end = start + strlen(start);
while (end > start && isspace((unsigned char)*(end - 1))) {
    * (--end) = '\0';
}

if (strlen(start) == 0) {
    free(line);
    return CSV_ERR_FORMAT;
}

// Преобразование строки в число
endptr = NULL;
value = strtol(start, &endptr, 10);
if (endptr == start || *endptr != '\0') {
    free(line);
    return CSV_ERR_FORMAT;
}

// Проверка переполнения массива
if (count >= max_len) {
    free(line);
    return CSV_ERR_TOO_LARGE;
}

// Сохранение значения в массив
array[count] = value;
count++;

if (comma == NULL) {
    break;
}
cursor = comma + 1;
}

free(line);

if (count == 0) {
    return CSV_ERR_EMPTY;
}

*out_len = count;
return CSV_OK;
}

// ФУНКЦИЯ ЗАПИСИ РЕЗУЛЬТАТОВ В CSV ФАЙЛ
// Параметры:
// filename - путь к файлу
// array - отсортированный массив
// len - размер массива
// shell_stats - статистика сортировки Шелла
// qsort_stats - статистика qsort
static int write_result_csv(const wchar_t* filename, const long* array, int len,
    const SortStats* shell_stats, const SortStats* qsort_stats)
{
    FILE* fp;
    int i;

```



```

// Открытие файла для записи
fp = _wopen(filename, L"w");
if (fp == NULL) {
    return -1;
}

// Запись массива в формате CSV
for (i = 0; i < len; i++) {
    fprintf(fp, "%ld", array[i]);
    if (i != len - 1) {
        fprintf(fp, ",");
    }
}
fprintf(fp, "\n");

// Запись метаданных и статистики
fprintf(fp, "size,%d\n", len);
fprintf(fp, "shell_comparisons,%ld\n", shell_stats->comparisons);
fprintf(fp, "shell_swaps,%ld\n", shell_stats->swaps);
fprintf(fp, "shell_time_seconds,%.6f\n", shell_stats->time_seconds);

if (qsort_stats != NULL) {
    fprintf(fp, "qsort_time_seconds,%.6f\n", qsort_stats->time_seconds);
}

fclose(fp);
return 0;
}

// АЛГОРИТМ СОРТИРОВКИ ШЕЛЛА
// Реализация сортировки Шелла с использованием последовательности Кнута (h = 3*h + 1)
// Параметры:
//   array - сортируемый массив
//   len - размер массива
//   stats - структура для сбора статистики
static void shell_sort(long* array, int len, SortStats* stats)
{
    long h;
    int i, j;
    long temp;
    clock_t start, end;

    // Инициализация счетчиков статистики
    stats->comparisons = 0;
    stats->swaps = 0;

    // Начало замера времени
    start = clock();

    // Вычисление начального шага по формуле Кнута
    h = 1;
    while (h < len / 3) {
        h = 3 * h + 1;
    }

    // Основной цикл сортировки с уменьшающимся шагом
    while (h >= 1) {
        // Сортировка вставками для подмассивов с шагом h
        for (i = h; i < len; i++) {
            temp = array[i];
            j = i;

```

```

        // Сдвиг элементов на позицию h влево, если они больше текущего
        while (j >= h) {
            stats->comparisons++;
            if (array[j - h] > temp) {
                array[j] = array[j - h];
                stats->swaps++;
                j -= h;
            }
            else {
                break;
            }
        }

        // Размещение элемента на итоговую позицию
        if (j != i) {
            array[j] = temp;
        }
    }

    // Уменьшение шага в 3 раза
    h /= 3;
}

// Завершение замера времени
end = clock();
stats->time_seconds = (double)(end - start) / CLOCKS_PER_SEC;
}

// ФУНКЦИЯ СРАВНЕНИЯ ДЛЯ QSORT
// Функция сравнения двух элементов типа long для стандартной qsort
static int compare_long(const void* a, const void* b)
{
    long va = *(const long*)a;
    long vb = *(const long*)b;

    if (va < vb) return -1;
    if (va > vb) return 1;
    return 0;
}

// ИЗМЕРЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ QSORT
// Запускает стандартную qsort и замеряет её время работы
static void measure_qsort(long* array, int len, SortStats* stats)
{
    clock_t start, end;

    start = clock();
    qsort(array, (size_t)len, sizeof(long), compare_long);
    end = clock();

    stats->time_seconds = (double)(end - start) / CLOCKS_PER_SEC;
    stats->comparisons = -1;
    stats->swaps = -1;
}

// КОПИРОВАНИЕ МАССИВА
// Копирует содержимое одного массива в другой
static void copy_array(const long* src, long* dst, int len)
{
    memcpy(dst, src, (size_t)len * sizeof(long));
}

// ГЕНЕРАЦИЯ ТЕСТОВЫХ ДАННЫХ
// Генерирует массив заданного типа

```

```

// Параметры:
// array - массив для заполнения
// len - размер массива
// kind - тип данных (из enum DATA_*)
// seed - начальное значение для генератора случайных чисел
static void generate_dataset(long* array, int len, int kind, unsigned int seed)
{
    int i;

    srand(seed);

    switch (kind) {
    case DATA_SORTED:
        // Отсортированный массив
        for (i = 0; i < len; i++) array[i] = i;
        break;

    case DATA_REVERSED:
        // Инвертированный массив
        for (i = 0; i < len; i++) array[i] = len - i;
        break;

    case DATA_REPEATED:
        // Массив с повторяющимися элементами (0-9)
        for (i = 0; i < len; i++) array[i] = rand() % 10;
        break;

    case DATA_RANDOM:
    default:
        // Случайный массив
        for (i = 0; i < len; i++) array[i] = rand() % (len * 10 + 1);
        break;
    }
}

// ИДЕНТИФИКАТОРЫ ЭЛЕМЕНТОВ УПРАВЛЕНИЯ GUI
#define ID_BTN_LOAD          101    // Кнопка загрузки CSV
#define ID_BTN_GENERATE      102    // Кнопка генерации данных
#define ID_BTN_SORT          103    // Кнопка сортировки
#define ID_BTN_SAVE          104    // Кнопка сохранения результата
#define ID_EDIT_SIZE         105    // Поле ввода размера массива
#define ID_COMBO_KIND        106    // Выпадающий список типа данных
#define ID_LIST_ARRAY        107    // Список для отображения массива
#define ID_EDIT_LOG          108    // Поле лога и статистики
#define ID_STATIC_SIZE       109    // Метка "Размер:"
#define ID_STATIC_KIND       110    // Метка типа данных

// ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ПРИЛОЖЕНИЯ
static long  g_array[MAX_SIZE];      // Основной массив данных
static long  g_array_copy[MAX_SIZE]; // Копия массива для сравнения алгоритмов
static int   g_len = 0;              // Текущий размер массива
static int   g_has_data = 0;         // Флаг наличия данных
static SortStats g_shell_stats;      // Статистика сортировки Шелла
static SortStats g_qsort_stats;      // Статистика qsort
static int   g_has_sorted = 0;       // Флаг выполнения сортировки

static HWND g_hList;                 // Дескриптор списка массива
static HWND g_hLog;                  // Дескриптор поля лога
static HWND g_hEditSize;             // Дескриптор поля размера
static HWND g_hComboKind;           // Дескриптор выпадающего списка

// ФУНКЦИЯ ВЫВОДА СООБЩЕНИЯ В ЛОГ
// Добавляет строку в поле лога
static void log_line(const char* text_utf8)

```

```

{
    wchar_t wbuf[1024];
    int len;

    // Преобразование UTF-8 в широкие символы
    len = MultiByteToWideChar(CP_UTF8, 0, text_utf8, -1, wbuf, (int)(sizeof(wbuf) /
sizeof(wchar_t)) - 2);
    if (len <= 0) {
        wbuf[0] = L'\0';
        len = 0;
    }
    wbuf[len] = L'\0';

    // Добавление текста в конец поля лога
    {
        int total_len = GetWindowTextLengthW(g_hLog);
        SendMessageW(g_hLog, EM_SETSEL, (WPARAM)total_len, (LPARAM)total_len);
        SendMessageW(g_hLog, EM_REPLACESEL, FALSE, (LPARAM)wbuf);
        SendMessageW(g_hLog, EM_REPLACESEL, FALSE, (LPARAM)L"\r\n");
    }
}

// ФУНКЦИЯ ФОРМАТИРОВАННОГО ВЫВОДА В ЛОГ
static void log_linef(const char* fmt, ...)
{
    char buf[512];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buf, sizeof(buf), fmt, args);
    va_end(args);
    log_line(buf);
}

// ПРЕОБРАЗОВАНИЕ UTF-8 В UNICODE
// Конвертирует строку из UTF-8 в широкие символы Windows
static wchar_t* utf8_to_wide(const char* s)
{
    int len = MultiByteToWideChar(CP_UTF8, 0, s, -1, NULL, 0);
    wchar_t* w;

    if (len <= 0) len = 1;
    w = (wchar_t*)malloc((size_t)len * sizeof(wchar_t));
    if (w == NULL) return NULL;
    if (MultiByteToWideChar(CP_UTF8, 0, s, -1, w, len) <= 0) {
        w[0] = L'\0';
    }
    return w;
}

// ФУНКЦИЯ ОТОБРАЖЕНИЯ ДИАЛОГОВОГО ОКНА
// Показывает MessageBox
static int msgbox_utf8(HWND hwnd, const char* text_utf8, const char* title_utf8, UINT
type)
{
    wchar_t* wtext = utf8_to_wide(text_utf8);
    wchar_t* wtitle = utf8_to_wide(title_utf8);
    int rc;

    rc = MessageBoxW(hwnd, wtext ? wtext : L"", wtitle ? wtitle : L"", type);

    free(wtext);
    free(wtitle);
    return rc;
}

```

```

// ОГРАНИЧЕНИЕ ОТОБРАЖЕНИЯ ЭЛЕМЕНТОВ В СПИСКЕ
#define LIST_DISPLAY_LIMIT 500

// ОБНОВЛЕНИЕ СПИСКА МАССИВА В ИНТЕРФЕЙСЕ
// Обновляет содержимое ListBox, отображая элементы массива
static void refresh_array_list(void)
{
    int i;
    wchar_t buf[64];
    int show;

    // Очистка списка
    SendMessageW(g_hList, LB_RESETCONTENT, 0, 0);

    if (!g_has_data) return;

    // Ограничение количества отображаемых элементов
    show = (g_len < LIST_DISPLAY_LIMIT) ? g_len : LIST_DISPLAY_LIMIT;

    // Добавление элементов в список
    for (i = 0; i < show; i++) {
        swprintf(buf, sizeof(buf) / sizeof(wchar_t), L "[%d] = %ld", i, g_array[i]);
        SendMessageW(g_hList, LB_ADDSTRING, 0, (LPARAM)buf);
    }

    // Сообщение о том, что показаны не все элементы
    if (show < g_len) {
        swprintf(buf, sizeof(buf) / sizeof(wchar_t), L"... показаны первые %d из %d элементов", show, g_len);
        SendMessageW(g_hList, LB_ADDSTRING, 0, (LPARAM)buf);
    }
}

// ОБРАБОТЧИК КНОПКИ "ЗАГРУЗИТЬ CSV"
// Открывает диалог выбора файла и загружает массив из CSV
static void do_load_csv(HWND hwnd)
{
    OPENFILENAMEW ofn;
    wchar_t filename[MAX_PATH] = L"";
    char filename_utf8[MAX_PATH * 2];
    int rc;

    // Инициализация структуры диалога открытия файла
    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"CSV файлы (*.csv)\\0*.csv\\0Все файлы (*.*)\\0*.*\\0";
    ofn.lpstrFile = filename;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;
    ofn.lpstrTitle = L"Выберите входной CSV-файл";

    // Показ диалога и проверка результата
    if (!GetOpenFileNameW(&ofn)) return;

    // Преобразование имени файла в UTF-8 для логирования
    WideCharToMultiByte(CP_UTF8, 0, filename, -1, filename_utf8,
sizeof(filename_utf8), NULL, NULL);

    // Чтение массива из файла
    rc = read_csv_array(filename, g_array, MAX_SIZE, &g_len);
    if (rc != CSV_OK) {
        char msg[300];

```

```

        sprintf(msg, "Ошибка загрузки файла:\n%s", csv_error_message(rc));
        msgbox_utf8(hwnd, msg, "Ошибка", MB_OK | MB_ICONERROR);
        g_has_data = 0;
        return;
    }

    // Обновление состояния и интерфейса
    g_has_data = 1;
    g_has_sorted = 0;
    refresh_array_list();
    log_linef("Загружено из файла: %s", filename_utf8);
    log_linef("Элементов: %d", g_len);
}

// ОБРАБОТЧИК КНОПКИ "СГЕНЕРИРОВАТЬ"
// Генерирует тестовый массив заданного размера и типа
static void do_generate(HWND hwnd)
{
    wchar_t size_text[32];
    int size;
    int kind_index;
    int kind;
    const char* kind_names[] = {
        "отсортированный", "инвертированный", "случайный", "с повторами"
    };

    // Получение размера из поля ввода
    GetWindowTextW(g_hEditSize, size_text, (int)(sizeof(size_text) /
sizeof(wchar_t)));
    size = _wtoi(size_text);

    // Проверка корректности размера
    if (size <= 0 || size > MAX_SIZE) {
        char msg[200];
        sprintf(msg, "Введите размер массива от 1 до %d.", MAX_SIZE);
        msgbox_utf8(hwnd, msg, "Некорректный размер", MB_OK | MB_ICONWARNING);
        return;
    }

    // Получение выбранного типа данных
    kind_index = (int)SendMessageW(g_hComboKind, CB_GETCURSEL, 0, 0);
    if (kind_index < 0) kind_index = 2;
    kind = kind_index + 1;

    // Генерация массива
    generate_dataset(g_array, size, kind, (unsigned int)time(NULL));
    g_len = size;
    g_has_data = 1;
    g_has_sorted = 0;

    // Обновление интерфейса
    refresh_array_list();
    log_linef("Сгенерирован массив (%s), размер %d", kind_names[kind_index], size);
}

// ОБРАБОТЧИК КНОПКИ "СОРТИРОВАТЬ"
// Выполняет сортировку Шелла и сравнение с qsort
static void do_sort(HWND hwnd)
{
    // Проверка наличия данных
    if (!g_has_data) {
        msgbox_utf8(hwnd, "Сначала загрузите или сгенерируйте данные.", "Нет данных",
MB_OK | MB_ICONWARNING);
        return;
    }

```

```

    }

    // Создание копии массива для сравнения алгоритмов
    copy_array(g_array, g_array_copy, g_len);

    // Сортировка массива алгоритмом Шелла
    shell_sort(g_array, g_len, &g_shell_stats);

    // Сортировка копии стандартной qsort
    measure_qsort(g_array_copy, g_len, &g_qsort_stats);
    g_has_sorted = 1;

    // Обновление списка отсортированных данных
    refresh_array_list();

    // Вывод статистики в лог
    log_linef("--- Сортировка выполнена (n = %d) ---", g_len);
    log_linef("Шелл: сравнений=%ld перестановок=%ld время=%.6f c",
        g_shell_stats.comparisons, g_shell_stats.swaps, g_shell_stats.time_seconds);
    log_linef("qsort: время=%.6f c", g_qsort_stats.time_seconds);

    // Вычисление и вывод отношения производительности
    if (g_qsort_stats.time_seconds > 0.0) {
        log_linef("Отношение времени (Шелл / qsort): %.2f",
            g_shell_stats.time_seconds / g_qsort_stats.time_seconds);
    }
}

// ОБРАБОТЧИК КНОПКИ "СОХРАНИТЬ CSV"
// Сохраняет результаты сортировки и статистику в CSV файл
static void do_save_csv(HWND hwnd)
{
    OPENFILENAMEW ofn;
    wchar_t filename[MAX_PATH] = L"result.csv";
    char filename_utf8[MAX_PATH * 2];

    // Проверка наличия данных
    if (!g_has_data) {
        msgbox_utf8(hwnd, "Нет данных для сохранения.", "Нет данных", MB_OK |
MB_ICONWARNING);
        return;
    }

    // Инициализация диалога сохранения файла
    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"CSV файлы (*.csv)\\0*.csv\\0Все файлы (*.*)\\0*.*\\0";
    ofn.lpstrFile = filename;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;
    ofn.lpstrDefExt = L"csv";
    ofn.lpstrTitle = L"Сохранить результат как";

    // Показ диалога
    if (!GetSaveFileNameW(&ofn)) return;

    // Преобразование имени файла
    WideCharToMultiByte(CP_UTF8, 0, filename, -1, filename_utf8,
sizeof(filename_utf8), NULL, NULL);

    // Запись результатов в файл
    {
        int rc;

```

```

    if (!g_has_sorted) {
        // Если сортировка не выполнялась, записываем только массив
        SortStats empty_stats;
        empty_stats.comparisons = 0;
        empty_stats.swaps = 0;
        empty_stats.time_seconds = 0.0;
        rc = write_result_csv(filename, g_array, g_len, &empty_stats, NULL);
    }
    else {
        // Запись массива и полной статистики
        rc = write_result_csv(filename, g_array, g_len, &g_shell_stats,
&g_qsort_stats);
    }

    if (rc != 0) {
        char msg[300];
        sprintf(msg, "Не удалось сохранить файл:\n%s", filename_utf8);
        msgbox_utf8(hwnd, msg, "Ошибка сохранения", MB_OK | MB_ICONERROR);
        return;
    }
}

log_linef("Результат сохранён в файл: %s", filename_utf8);
}

// ОБРАБОТЧИК СООБЩЕНИЙ ОКНА
// Главная функция обработки оконных сообщений Windows
LRESULT CALLBACK WindowProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg) {
    case WM_CREATE: {
        // Создание всех элементов управления при создании окна
        HFONT hFont = (HFONT)GetStockObject(DEFAULT_GUI_FONT);
        HWND child;

        // Кнопка загрузки CSV файла
        CreateWindowW(L"BUTTON", L"Загрузить CSV...",
            WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
            20, 20, 150, 30, hwnd, (HMENU)ID_BTN_LOAD, NULL, NULL);

        // Метка для поля размера
        CreateWindowW(L"STATIC", L"Размер:",
            WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
            190, 20, 50, 20, hwnd, (HMENU)ID_STATIC_SIZE, NULL, NULL);

        // Поле ввода размера массива
        g_hEditSize = CreateWindowW(L"EDIT", L"1000",
            WS_VISIBLE | WS_CHILD | WS_BORDER | WS_CLIPSIBLINGS,
            245, 18, 70, 24, hwnd, (HMENU)ID_EDIT_SIZE, NULL, NULL);

        // Выпадающий список типа данных
        g_hComboKind = CreateWindowW(L"COMBOBOX", L"",
            WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS |
            CBS_DROPDOWNLIST | CBS_HASSTRINGS,
            325, 18, 160, 150, hwnd, (HMENU)ID_COMBO_KIND, NULL, NULL);

        SendMessageW(g_hComboKind, WM_SETFONT, (WPARAM)hFont, FALSE);
        SendMessageW(g_hComboKind, CB_ADDSTRING, 0, (LPARAM)L"Отсортированный");
        SendMessageW(g_hComboKind, CB_ADDSTRING, 0, (LPARAM)L"Инвертированный");
        SendMessageW(g_hComboKind, CB_ADDSTRING, 0, (LPARAM)L"Случайный");
        SendMessageW(g_hComboKind, CB_ADDSTRING, 0, (LPARAM)L"С повторами");
        SendMessageW(g_hComboKind, CB_SETCURSEL, 2, 0);
    }
    }
}

```



```

// Кнопка генерации данных
CreateWindowW(L"BUTTON", L"Сгенерировать",
    WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
    495, 20, 130, 30, hwnd, (HMENU)ID_BTN_GENERATE, NULL, NULL);

// Кнопка запуска сортировки
CreateWindowW(L"BUTTON", L"Сортировать (Шелл + qsort)",
    WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
    20, 60, 220, 32, hwnd, (HMENU)ID_BTN_SORT, NULL, NULL);

// Кнопка сохранения результатов
CreateWindowW(L"BUTTON", L"Сохранить CSV...",
    WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
    250, 60, 150, 32, hwnd, (HMENU)ID_BTN_SAVE, NULL, NULL);

// Метка списка массива
CreateWindowW(L"STATIC", L"Массив:",
    WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
    20, 105, 100, 20, hwnd, NULL, NULL, NULL);

// Список для отображения элементов массива
g_hList = CreateWindowW(L"LISTBOX", L"",
    WS_VISIBLE | WS_CHILD | WS_BORDER | WS_VSCROLL | WS_CLIPSIBLINGS |
LBS_NOTIFY,
    20, 128, 320, 380, hwnd, (HMENU)ID_LIST_ARRAY, NULL, NULL);

// Метка поля лога
CreateWindowW(L"STATIC", L"Лог и статистика:",
    WS_VISIBLE | WS_CHILD | WS_CLIPSIBLINGS,
    360, 105, 150, 20, hwnd, NULL, NULL, NULL);

// Многострочное поле для вывода лога и статистики
g_hLog = CreateWindowW(L"EDIT", L"",
    WS_VISIBLE | WS_CHILD | WS_BORDER | WS_VSCROLL | WS_CLIPSIBLINGS |
    ES_MULTILINE | ES_AUTOVSCROLL | ES_READONLY,
    360, 128, 320, 380, hwnd, (HMENU)ID_EDIT_LOG, NULL, NULL);

// Установка шрифта для всех элементов управления
child = GetWindow(hwnd, GW_CHILD);
while (child != NULL) {
    SendMessageW(child, WM_SETFONT, (WPARAM)hFont, TRUE);
    child = GetWindow(child, GW_HWNDNEXT);
}

// Начальное сообщение в логе
log_line("Готово к работе. Загрузите CSV-файл или сгенерируйте тестовые
данные.");
break;
}

case WM_COMMAND: {
    // Обработка команд от элементов управления
    int id = LOWORD(wParam);
    switch (id) {
        case ID_BTN_LOAD:      do_load_csv(hwnd); break;
        case ID_BTN_GENERATE:  do_generate(hwnd); break;
        case ID_BTN_SORT:      do_sort(hwnd); break;
        case ID_BTN_SAVE:      do_save_csv(hwnd); break;
    }
    break;
}

case WM_DESTROY:

```

```

        // Завершение работы приложения при закрытии окна
        PostQuitMessage(0);
        break;

default:
    // Обработка всех остальных сообщений стандартным обработчиком
    return DefWindowProcW(hwnd, msg, wParam, lParam);
}

return 0;
}

// ТОЧКА ВХОДА В ПРИЛОЖЕНИЕ
// Главная функция Windows-приложения
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int
nCmdShow)
{
    const wchar_t CLASS_NAME[] = L"ShellSortWindowClass";
    WNDCLASSW wc;
    HWND hwnd;
    MSG msg;

    (void)hPrevInstance;
    (void)lpCmdLine;

    // Включение поддержки DPI для корректного отображения на разных разрешениях
    SetProcessDpiAwarenessContext(DPI_AWARENESS_CONTEXT_PER_MONITOR_AWARE_V2);

    // Регистрация класса окна
    ZeroMemory(&wc, sizeof(wc));
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_BTNFACE + 1);
    wc.hCursor = LoadCursorW(NULL, IDC_ARROW);

    RegisterClassW(&wc);

    // Создание главного окна приложения
    hwnd = CreateWindowExW(
        0,
        CLASS_NAME,
        L"Сортировка Шелла",
        (WS_OVERLAPPEDWINDOW & ~WS_THICKFRAME & ~WS_MAXIMIZEBOX) | WS_CLIPCHILDREN,
        CW_USEDEFAULT, CW_USEDEFAULT, 720, 560,
        NULL, NULL, hInstance, NULL
    );

    if (hwnd == NULL) return 0;

    // Отображение окна
    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    // Главный цикл обработки сообщений
    while (GetMessageW(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return 0;
}

```